

# Informe de rendimiento: P3ERP en localhost con AWS RDS y Cognito

Fecha: 2 de julio de 2026. Entorno analizado: Next.js 15.5.19, Node 20.10.0, localhost, PostgreSQL en AWS RDS, Cognito, código actual del repo.

**Cuello principal observado:**

no es la consulta SQL caliente, sino compilación/dev server, middleware Cognito, arranque con Sequelize y demasiada carga en cliente.

**RDS caliente:**

consultas representativas con pool reutilizado rondan 20-22 ms. La conexión fría cuesta 170-275 ms.

**Next dev:**

primer golpe a APIs tarda 0.5-3.6 s por compilación. Luego baja a 40-165 ms.

**Build:**

falló tras 96.9 s por certificado RDS al prerenderizar /gm/nuevo . También aparecen warnings de módulos Node en Edge Runtime por Cognito.

## Pruebas realizadas

| Prueba                     | Objetivo                                                        | Resultado                                                                                                                                              |
|----------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inspección de arquitectura | Localizar auth, DB, layouts, fetches cliente, páginas críticas. | Middleware protege todas las rutas no API; dashboard tiene layout cliente; hay bootstrap Sequelize global; muchas páginas cargan datos por useEffect . |
| RDS conexión fría          | Medir coste de abrir conexión TLS nueva a RDS.                  | SELECT 1 con pool nuevo: min 170 ms, p50 174 ms, p90 275 ms, media 195 ms.                                                                             |
| RDS con pool caliente      | Medir SQL real sin coste de conexión.                           | Consultas de cuentas/contactos/agentes/detalle: p50 20-22 ms.                                                                                          |
| Next dev APIs              | Medir coste de compilación y ejecución API interna.             | Primer golpe: hasta 3.6 s. Golpes calientes: ~41-165 ms.                                                                                               |
| Build producción           | Detectar errores de prerender y coste de compilación.           | Compilación OK en 58 s, pero build falla al generar páginas por RDS SSL: SELF_SIGNED_CERT_IN_CHAIN .                                                   |
| Análisis estático          | Cuantificar superficie cliente.                                 | 220 archivos TS/TSX/JS, 20.275 líneas, 76 archivos use client , 54 archivos con useEffect , 15 servicios con API client.                               |

## Resultados medidos

### RDS directo

| Consulta                       | Filas | p50                 | p90    | Conclusión                                   |
|--------------------------------|-------|---------------------|--------|----------------------------------------------|
| SELECT 1, pool caliente        | 1     | 21 ms               | 22 ms  | Latencia base de RDS desde localhost.        |
| cuentas_db ORDER BY created_at | 3     | 22 ms               | 24 ms  | No es lento con dataset actual.              |
| agentes_db ORDER BY nombre     | 7     | 20 ms               | 22 ms  | Correcto.                                    |
| contactos_db ORDER BY nombre   | 8     | 21 ms               | 23 ms  | Correcto.                                    |
| Detalle cuenta                 | 1     | 22 ms               | 23 ms  | Correcto.                                    |
| GM detalle + contactos         | 1 + 2 | ~22 ms por consulta | ~23 ms | Mejorable si se agrupa, pero no crítico hoy. |

## Next dev: APIs internas

| Endpoint                    | Primer golpe | Golpes calientes | Lectura                                       |
|-----------------------------|--------------|------------------|-----------------------------------------------|
| /api/v1/comercial/cuentas   | 3627 ms      | 62, 439, 84 ms   | Primer golpe dominado por compilación dev.    |
| /api/v1/admin/agentes       | 929 ms       | 101-102 ms       | Se llama además de cuentas en la pantalla.    |
| /api/v1/comercial/contactos | 536 ms       | 41-110 ms        | Caliente aceptable.                           |
| /api/v1/gm/cuentas          | 741 ms       | 55-106 ms        | Caliente aceptable.                           |
| /api/v1/gm/cuentas/62500001 | 1814 ms      | 120-165 ms       | Primer golpe compila ruta dinámica y repo GM. |

## Hallazgos principales

- Bootstrap Sequelize global en instrumentation-node.js**. En cada arranque de Next se ejecuta conexión y `sequelize.sync()`, aunque la app nueva usa sobre todo `pg.Pool`. Esto añade latencia de arranque, ruido de logs, carga de Sequelize y warnings por dialectos dinámicos. También infla compilación: los logs muestran imports de dialectos SQLite/MariaDB aunque se use PostgreSQL.
- Middleware Cognito costoso en cada navegación**. El matcher aplica a todas las rutas no API, incluidas `/gm`. El middleware hace un `fetch` interno a `/api/validate-token` en cada request de página. Eso añade una vuelta HTTP local, verificación de tokens y potencial acceso a Cognito si hay refresh. Además el build avisa que `aws-jwt-verify` carga módulos Node no soportados en Edge Runtime.
- Demasiado trabajo cliente**. Muchas páginas críticas son `use client` y cargan datos en `useEffect`. Esto retrasa contenido hasta después de hidratar JS. Ejemplo: `/dashboard/comercial/cuentas` pide cuentas y agentes en dos requests cliente separados.
- Filtros sin debounce**. En cuentas, cada tecla cambia estado y dispara `CuentaService.getCuentas`. Con RDS remoto y APIs internas, escribir rápido puede lanzar muchas llamadas en paralelo.
- Consultas poco selectivas para escalar**. Hay varios `SELECT *`, filtros `ILIKE '%texto%'` y un filtro sobre `datos_comerciales::text`. Con pocos datos no duele; con miles de cuentas/contactos sí.
- Contactos comercial todavía no consume RDS en la lista**. La página `/dashboard/comercial/contactos` parte de un array vacío, por lo que no mide carga real ni aprovecha `ContactoService`. Cuando se conecte a RDS debería evitar traer todo y filtrar todo en cliente.
- Build de producción roto por SSL RDS en prerender**. `/gm/nuevo` consulta RDS durante prerender y falla con `SELF_SIGNED_CERT_IN_CHAIN`. En dev se tolera `rejectUnauthorized: false` en `pgClient`, pero en build/production cambia el comportamiento.
- Build lento y warnings de hooks**. `next build` compiló en 58 s y acabó fallando a los 96.9 s. Hay warnings de hooks condicionales y dependencias de `useEffect` que pueden causar rerenders o bugs.

## Cuellos por pantalla

| Pantalla        | Cuello más probable                                                                                                                          | Acción recomendada                                                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cuentas         | Dos requests cliente iniciales: cuentas + agentes. Filtros lanzan request por tecla. Paginación solo en cliente.                             | Server Component o endpoint combinado <code>cuentas_con_agente</code> ; debounce 300 ms; paginación SQL con <code>LIMIT/OFFSET</code> ; no <code>SELECT *</code> . |
| Contactos       | Lista aún no conectada a RDS; al migrarla, riesgo de traer todos los contactos y filtrar en cliente.                                         | Crear endpoint con filtros SQL e índices; paginación real; cargar solo columnas de tabla.                                                                          |
| Detalle cuenta  | Varias pestañas hacen fetch al activarse: comentarios, contactos, propuestas, etc. Correcto funcionalmente, pero puede sentirse fragmentado. | Prefetch de datos de pestañas más usadas, cache SWR/React Query o Server Component con datos iniciales.                                                            |
| GM              | Ahora lee RDS server-side para detalle, pero el middleware lo protege y redirige si no hay cookie. Home GM carga por cliente.                | Si GM debe ser rápido, hacerlo server-render con datos iniciales o excluirlo del middleware si no requiere sesión.                                                 |
| Barra izquierda | No es el cuello principal. Es cliente y recalcula apertura por pathname, pero es pequeño.                                                    | Puede simplificarse declarando estructura de menú y usando CSS/estado mínimo. El coste real viene del middleware al navegar, no del componente.                    |

## Plan de optimización recomendado

### Prioridad 1: quitar lastre global y arreglar auth

- Eliminar o condicionar `instrumentation-node.js` para que no haga `sequelize.sync()` en dev normal ni build. Si ya usas `pg.Pool` , retirar Sequelize o aislarlo detrás de una variable `ENABLE_SEQUELIZE_BOOTSTRAP=true` .
- Reescribir middleware para no hacer `fetch('/api/validate-token')` por navegación. Verificar el JWT directamente en middleware con una librería compatible Edge, o mover auth a Node en route handlers/layout protegido.
- Evitar refresh token desde middleware Edge. Refrescar desde cliente/API dedicada cuando haga falta.
- Configurar correctamente SSL RDS para producción/build: usar CA de AWS o ajustar `pgClient` para build local. Ahora el build falla por certificado.

### Prioridad 2: reducir waterfalls de datos

- En cuentas, devolver agente ya resuelto desde SQL: `LEFT JOIN agentes_db` . Así se elimina `/api/v1/admin/agentes` en la carga inicial.
- Convertir páginas de lista a Server Components donde sea posible, pasando datos iniciales a una tabla cliente solo para interacción.
- Implementar debounce en filtros y cancelar requests anteriores con `AbortController` .
- Paginación y filtros en servidor: `LIMIT` , `OFFSET` , `total_count` .

### Prioridad 3: preparar RDS para volumen

- Índices recomendados: `contactos_db(nombre_contacto)` , `contactos_db(apellidos_contacto)` , `contactos_db(nombre_empresa)` , `contactos_db(email_contacto)` , `cuentas_db(nombre_empresa)` ya existe, revisar índice para `id_cuenta` ya cubierto por PK.
- Para búsquedas parciales `%texto%` , valorar extensión `pg_trgm` e índices GIN trigram en nombres/emails.
- Evitar `datos_comerciales::text ILIKE` ; extraer teléfono principal a columna real o índice expression si se mantiene JSONB.
- No usar `to_date()` en ORDER BY sobre columnas text si crecerá el dato; migrar fechas a `date` .

### Prioridad 4: limpieza de frontend

- Reducir archivos `use client`. Dejar `client` solo en componentes con estado/eventos.
- Revisar warnings de hooks del build: hay hooks condicionales que pueden provocar comportamiento errático y rerenders.
- Eliminar imports/código legado de fichajes y servicios antiguos que todavía aparecen en `services`.
- Considerar `React Query/SWR` para cachear listas como agentes, cuentas, contactos y evitar `refetch` al volver por la barra lateral.

## Notas sobre desarrollo local

---

- Next dev recompila rutas bajo demanda: el primer acceso puede tardar segundos aunque producción sea más rápida.
- OneDrive puede empeorar I/O y ya ha provocado bloqueos en `.next/trace`. Para desarrollo, mover el repo fuera de OneDrive suele mejorar mucho la experiencia.
- `node_modules` pesa ~496 MB y `.next` tras build fallido ~378 MB. La carpeta de build puede bloquearse en OneDrive.

## Conclusión

---

Con los datos actuales, RDS no es el problema principal una vez reutilizas conexión: las queries calientes son rápidas. La lentitud percibida viene de una combinación de Next dev, middleware Cognito en cada navegación, bootstrap Sequelize global, build/prerender consultando RDS y un patrón frontend de cargar datos después de hidratar. Las mejores mejoras inmediatas son quitar/condicionar Sequelize bootstrap, simplificar auth middleware, combinar endpoints de lista y mover filtros/paginación al servidor.